Tarea 3: Diseño de un juego de luces mejorado

Project 3: Design of an upgraded lighting effect

En este proyecto realizaremos un rediseño del juego de luces del Proyecto 1 utilizando máquinas de estados, y le añadiremos funcionalidad adicional para control de velocidad mediante los botones de la placa de evaluación.

In this project we will redesign the lighting effect from Project 1 using state machines, and we will add a new function for speed control with the push buttons on the evaluation board.

Objetivos

Goals

- Describir máquinas de estados finitos en Verilog
- Diseñar máquinas de estados de Moore y de Mealy
- Diseñar circuitos para la lectura de botones pulsadores
- Crear *testbenches* estructurados mediante *tasks*

- Describing finite state machines in Verilog
- Designing Moore and Mealy state machines
- Designing circuits for the readout of push buttons
- Creating structured testbenches using *tasks*

Tareas a realizar

Work to be done

- Apartado 1: Circuito y testbench.
- Apartado 2: Circuito y testbench.
- Apartado 3: Circuito, testbench, y prueba en placa.
- Informe del proyecto.

Los testbench de este proyecto deberán estar bien estructurados (uso adecuado de *tasks*, temporización basada en cuenta de flancos en lugar de retardos fijos, etc.)

- Part 1: Circuit and testbench.
- Part 2: Circuit and testbench.
- Part 3: Circuit, testbench, and test on board.
- Project report.

Testbenches should be well structured (appropriate use of tasks, timing based on edge counting instead of fixed delays, etc.)

Estructura del proyecto

Project structure

El proyecto se realizará en tres pasos incrementales en los que iremos añadiendo bloques al diseño. En primer lugar reharemos el Proyecto 1 mediante máquinas de estado, a continuación implementaremos un circuito que realice la lectura de la pulsación de botones para controlar la velocidad, y por último integraremos todo en un único módulo.

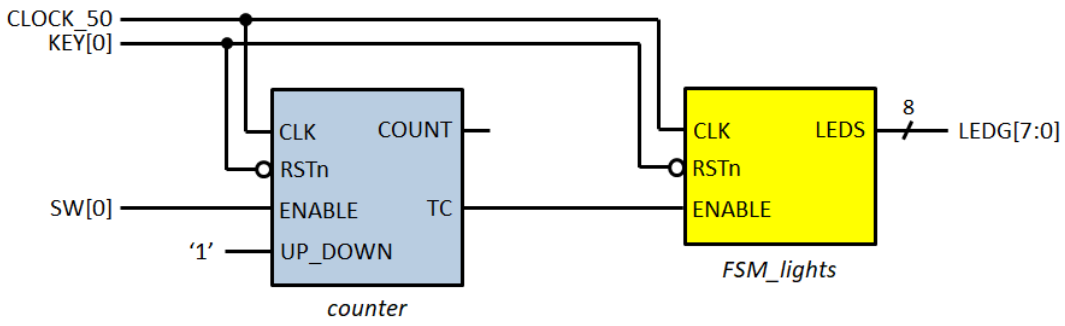
The project will be carried out in three incremental steps in which different blocks will keep being added to the design. First of all, we will remake Project 1 using finite state machines. Then, we will implement a circuit for the readout of push buttons that will allow us to the speed of the lights. Finally, we will integrate all blocks in a single module.

Apartado 1: Juego de luces mediante máquina de estados

Part 1: Lighting effect using a state machine

En el primer apartado, volveremos a implementar el juego de luces del Proyecto 1 utilizando una máquina de estados. El esquema del circuito será el siguiente.

As a first step, we will re-implement the lighting effect from Project 1 using a finite state machine. The circuit schematic will be as follows.



El bloque ***FSM_lights*** es una máquina de estados de **Moore,** tipo **Medvedev** que hará el papel del contador Johnson y el decodificador del Proyecto 1. Debe ser una máquina de **14 estados** que genere la secuencia de luces especificada, que volvemos a incluir aquí.

The ***FSM_lights*** block is a **Moore** state machine (**Medvedev** type) that will play the role of the Johnson counter and the decoder from Project 1. It should be a state machine with **14 states** that generates the specified light sequence, as shown below.

Ciclos	LEDG7	LEDG6	LEDG5	LEDG4	LEDG3	LEDG2	LEDG1	LEDG0
1	■							
2		■						
3			■					
4				■				
5					■			
6						■		
7							■	
8								■
9							■	
10						■		
11					■			
12				■				
13			■					
14		■						

Se ha añadido también un contador (***counter***) que actúa como divisor de frecuencia. Puede utilizarse el mismo contador que en proyectos anteriores. En este caso, para la prueba en placa utilizaremos un módulo de 5000000 para conseguir que la señal TC generada tenga una frecuencia de 10 Hz.

Additionally, there is a counter that acts as a frequency divider. You may use the same counter module as in previous projects. In this case, we will use a modulus of 5000000 for on-board testing, so that the generated TC signal has a frequency of 10 Hz.

Describe el diseño en Verilog y compílalo. ¿Ha detectado Quartus la máquina de estados? ¿Cuál es la codificación de estados elegida por Quartus? No olvides incluir los resultados del *State Machine Viewer* en tu informe de prácticas.

Realiza un testbench para validar su funcionamiento en simulación. Puedes basarte en el realizado en el Proyecto 1, pero ten en cuenta que en este caso el testbench deberá estar bien estructurado (uso adecuado de *tasks*, temporización basada en cuenta de flancos en lugar de retardos fijos, etc.).

Prueba el diseño también en la placa de evaluación para comprobar el movimiento de las luces. Usaremos el pulsador KEY[0] como señal de reset y el interruptor SW[0] como señal de habilitación que activa o pausa el movimiento de las luces. Recuerda la técnica utilizada en la Tarea 1 para conseguir utilizar distintos valores del divisor de frecuencia para la simulación y para la prueba en placa sin necesidad de modificar el código.

Describe the design in Verilog and compile it. Has Quartus detected the state machine? What is the state encoding selected by Quartus? Don't forget to include the *State Machine Viewer* results in your project report.

Write a testbench in order to validate the design in simulation. It may be based on the one you used for Project 1, but keep in mind that the testbench is now required to be well structured (appropriate use of tasks, timing based on edge counting instead of fixed delays, etc.

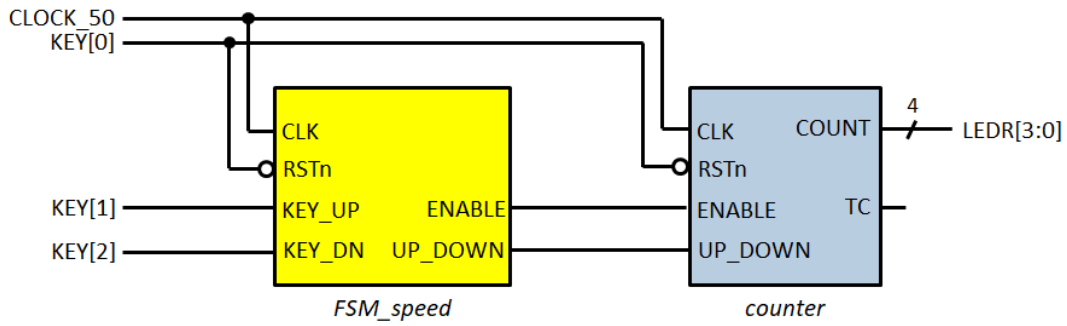
Run the design on the evaluation board as well to test the lighing effect. We will use button KEY[0] as the reset signal and switch SW[0] as the enable signal that activates or pauses the movement of the lights. Remember the technique used in Projects 1 to apply different values of the frequency divider in simulation and in physical verification with no need to modify the code.

Apartado 2: Control de velocidad

Part 2: Speed control

En el segundo apartado vamos a implementar el subsistema de control de velocidad. El diseño final tendrá dos botones pulsadores como entradas, que sirven para aumentar y disminuir la velocidad de movimiento, respectivamente. Este bloque de control consiste en un contador cuyo valor debe aumentar o disminuir en uno con cada pulsación del botón correspondiente. Una máquina de estados se encarga de comunicar los botones con el contador, traduciendo cada pulsación a las señales de control adecuadas.

In the second part of the project, we will implement a subsystem for speed control. The final design will have two push buttons as inputs, whose role is to increase and decrease movement speed, respectively. This control block consists in a counter whose value must increase or decrease by one with every push of the corresponding button. A finite state machine will be in charge of communicating the buttons with the counter, translating each push to the appropriate control signals.

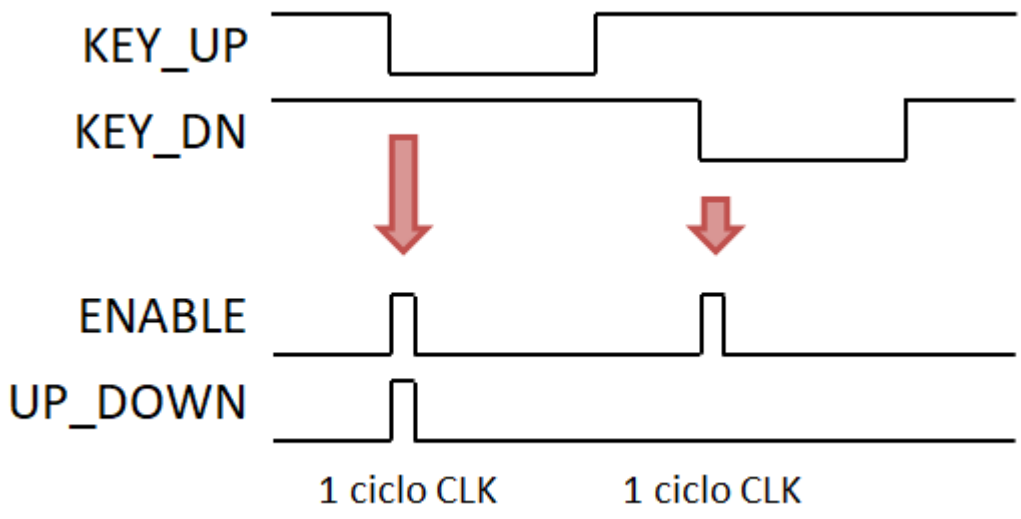


El contador *counter* será el mismo que en diseños anteriores, eligiendo en este caso un módulo de 16. El bloque ***FSM_speed*** es una máquina de estados de **Mealy** que debe generar las señales de control ENABLE y UP_DOWN para el contador, de tal manera que cuando se pulse el botón KEY[1], conectado a KEY_UP, el contador aumente su valor en 1, y cuando se pulse el botón KEY[2], conectado a KEY_DN, el contador disminuya su valor en 1. Ten en cuenta que:

- Los botones son activos a nivel bajo, es decir, la señal correspondiente vale 0 cuando el botón está pulsado y 1 cuando no lo está.
- El contador debe variar su valor sólo una vez por cada pulsación del botón, aunque éste se mantenga pulsado durante varios ciclos de reloj.

We will use the same counter as in previous designs, choosing a modulus of 16. The ***FSM_speed*** block is a **Mealy** state machine that must generate the ENABLE and UP_DOWN control signals for the counter, in such a way that when we press the button KEY[1], connected to KEY_UP, the counter increases its value by 1, and when we press the button KEY[2], connected to KEY_DN, the counter decreases its value by 1. Keep in mind that:

- Buttons are active low, that is, the signal from the button is 0 when the button is pressed and 1 when it's released.
- The counter must change its value only once per button push, even if the button remains pressed for several clock cycles.



Describe el diseño en Verilog y realiza un testbench para verificar el funcionamiento del diseño mediante simulación. Este testbench deberá comprobar la respuesta frente a distintos casos de test, **incluyendo pulsaciones realistas de botones**, y deberá estar estructurado de manera adecuada, preferentemente mediante el uso de *tasks* de Verilog. En el informe se valorará que se **incluyan las variables de estado** (state/next_state si procede) en las capturas de simulación. Prueba el diseño también en la placa de evaluación. Para ello podemos conectar la salida de 4 bits del contador a los cuatro LEDs rojos de la placa LEDR[3:0] para poder observar su valor en binario.

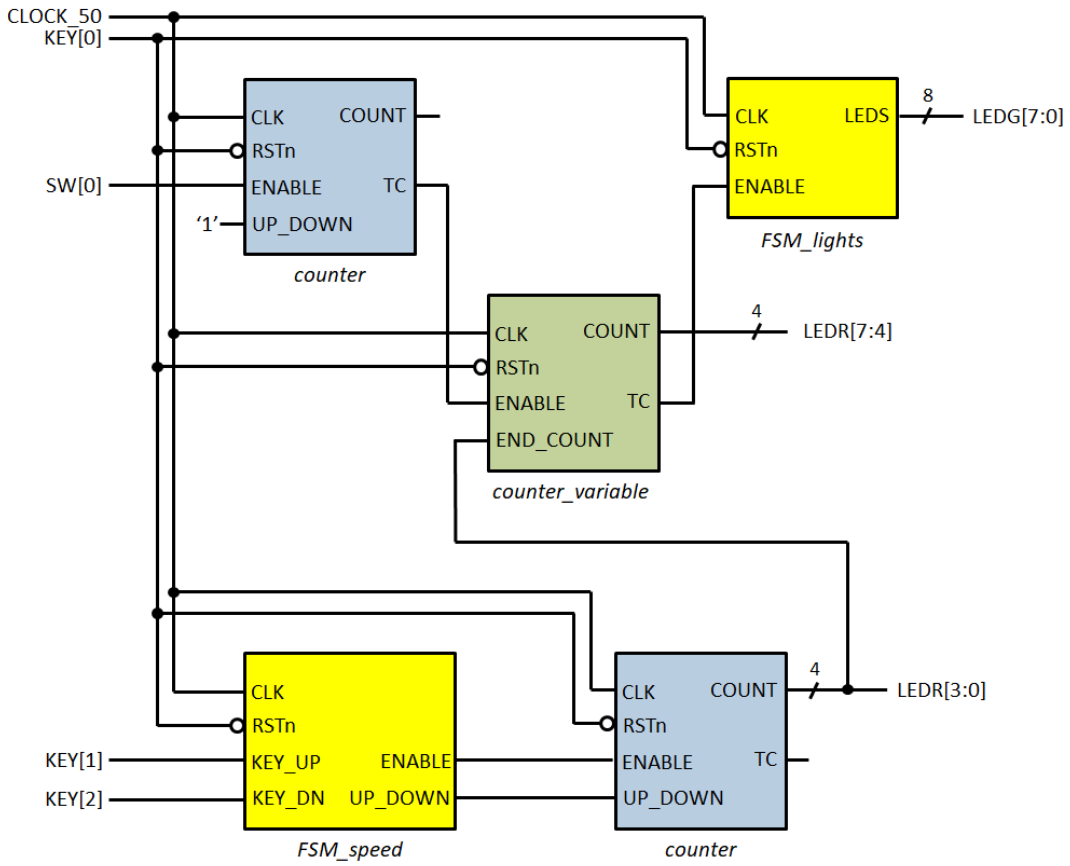
Describe the design in Verilog and write a testbench to verify its operation via simulation. The testbench must check the design's response to different test cases, **including realistic button presses**, and should be structured in an appropriate way, preferably around the use of Verilog *tasks*. The state register signals (state/ next_state if it exists) should be included in the waveform figures in the report. Test the design on the evaluation board as well. To do that, we can connect the counter's 4-bit output to the four red LEDs LEDR[3:0] on the board so we can see its value in binary format.

Apartado 3: Integración del diseño completo

Part 3: Integrating the complete design

Por último, integraremos todos los bloques realizados en los apartados anteriores para obtener el diseño completo. El esquema es el que se muestra en la figura siguiente.

To finish this project, we will integrate all blocks from previous steps in order to obtain the complete design. The schematic is shown on the next figure.


CLOCK_50
KEY[0]
SW[0]
'1'
CLK
RSTn
ENABLE
UP_DOWN
COUNT
TC
counter
CLK
RSTn
ENABLE
LEDS
8
LEDG[7:0]
FSM_lights
CLK
RSTn
ENABLE
END_COUNT
COUNT
TC
4
LEDR[7:4]
counter_variable
CLK
RSTn
KEY_UP
KEY_DN
ENABLE
UP_DOWN
KEY[1]
KEY[2]
FSM_speed
CLK
RSTn
ENABLE
UP_DOWN
COUNT
TC
4
LEDR[3:0]
counter

En esencia, se ha modificado el diseño del apartado 1 añadiendo un divisor de frecuencia adicional implementado en el módulo ***counter_variable***. Este módulo es un contador de 4 bits cuyo módulo es variable; es decir: *no es un parámetro*, que queda fijo una vez compilado el diseño, *sino que es una de las señales de entrada del módulo*, cuyo valor puede variar en funcionamiento. Su entrada END_COUNT indica el valor máximo que el contador puede contener, es decir, el módulo del contador es igual a END_COUNT + 1. Utilizaremos un valor de END_COUNT entre 0 y 15 generado por el subsistema de control de velocidad del apartado 2. Así, cuanto mayor sea este valor, mayor será el módulo de *counter_variable* y menor será la velocidad a la que se mueven las luces.

Os proporcionamos a continuación el código Verilog completo del módulo *counter_variable*:

Essentially, this modifies the design from part 1 by adding an additional frequency divider implemented in module ***counter_variable***. This module is a 4-bit counter whose modulus is variable; that is: *it's not a parameter*, whose value would be fixed once the design is compiled, *but it's one of the module's input signals*, whose value may change during operation. Its input END_COUNT indicates the maximum count value that the counter can hold, that is, the counter's modulus is equal to END_COUNT + 1. We will use a value between 0 and 15 for END_COUNT, generated by the speed control subsystem from part 2. Thus, the higher this value, the higher the modulus of *counter_variable* and the lower the speed at which the lights move.

Here, we provide the complete Verilog code for the *counter_variable* module:

```
module counter_variable (CLK, RSTn, ENABLE, END_COUNT, COUNT, TC);

  parameter width = 4;

  input CLK, RSTn, ENABLE;
  input [width-1:0] END_COUNT;
  output reg [width-1:0] COUNT;
  output TC;

  always @(posedge CLK or negedge RSTn)
  if (~RSTn)
    COUNT <= 0;
  else if (ENABLE)
  begin
    if (COUNT >= END_COUNT)
      COUNT <= 0;
    else
      COUNT <= COUNT + 1;
  end

  assign TC = ((COUNT == END_COUNT) && (ENABLE == 1)) ? 1 : 0;

endmodule
```

Describe el diseño completo en Verilog y verifícalo mediante un testbench. Puedes basarte en los de los dos apartados anteriores para simular distintos casos de pulsación de botones así como el efecto de la señal de habilitación. Prueba el diseño también en la placa de evaluación. En este caso conectaremos la salida del contador variable a los cuatro LEDs rojos LEDR[7:4] para poder observar su valor al mismo tiempo que el valor del contador del apartado 2.

Write the Verilog description of the complete design and test it using a testbench. You may use the testbenches from the previous steps of the project as inspiration to simulate different cases of button presses, as well as the effect of the enable signal. Try the design on the evaluation board as well. In this case, the output of the variable counter will be connected to the four red LEDs LEDR[7:4] so we can observe its value at the same time as the value of the counter from part 2.

En todos los apartados utilizaremos las mismas asignaciones de pines para las distintas entradas y salidas. La siguiente tabla recoge la información necesaria.

The same pin assignments for inputs and outputs will be used throughout all steps in the project. The following table collects the necessary information.

Nombre Puerto	Pin FPGA	Nombre Puerto	Pin FPGA	Nombre Puerto	Pin FPGA
CLOCK_50	Y2	LEDG[0]	E21	LEDR[0]	G19
KEY[0]	M23	LEDG[1]	E22	LEDR[1]	F19
KEY[1]	M21	LEDG[2]	E25	LEDR[2]	E19
KEY[2]	N21	LEDG[3]	E24	LEDR[3]	F21
KEY[3]	R24	LEDG[4]	H21	LEDR[4]	F18
SW[0]	AB28	LEDG[5]	G20	LEDR[5]	E18
SW[1]	AC28	LEDG[6]	G22	LEDR[6]	J19
SW[2]	AC27	LEDG[7]	G21	LEDR[7]	H19